

Computational Complexity

Complexity of some algorithms can be expressed as a small polynomial. For example:

- Linear Search: $O(n)$
- Matrix Multiplication: $O(n^{2.81})$
- Sorting: $O(n \log n)$

A problem X is called polynomial if any instance of X can be solved by an algorithm that runs in $O(n^m)$ time, where m is a constant and n is the size of input.

Definition: P is the class of all decision problems that can be solved by deterministic algorithms in polynomial time.

Definition: NP is the class of all decision problems solvable by nondeterministic algorithms in polynomial time.

It is easy to see that $P \subseteq NP$. It has never been proven whether $P = NP$ or $P \neq NP$. However, Cook has shown that if the Satisfiability problem (SAT) is in P , then $P = NP$.

Nondeterministic Algorithms

1. At some step, a choice might be made — we can't tell by looking at the program which choice will be made, but it will always be the right one.
2. Program leads to a "success" or a "failure" statement. If there is a sequence of choices that can lead to success, they will be taken. In fact, the shortest such sequence will be taken. Only if there is no way to reach success will failure be reached.
3. Time for "choice", "success" or "failure" is $O(1)$.
4. Nondeterministic programs are executed by "nondeterministic machines" that do not exist.

We will look at "nondeterministic decision algorithms" — ND algorithms that *make some decision* and report it with the "success" or "failure" output. The output of a decision algorithm (ND or D) is uniquely determined by its input and the algorithm itself.

Nondeterministic Search Algorithm

```

procedure NDSearch(A, n, x)
  j ← choice(1, n) // choose a subscript from 1 to n
  if (A[j] == x) then print(j); success; endif
  print("Nope")
  failure
end

```

ND machine wants to reach success — if x is in the array, then the correct value will be chosen for j .

Nondeterministic Sort

```

procedure NDSort(A, n)
  integer B[n]
  B  $\leftarrow$  0 // B is initialized to zero
  for i  $\leftarrow$  1 to n do
    j  $\leftarrow$  choice(1, n)
    if (B[j]  $\neq$  0) then failure // makes sure an empty spot in B
                                // is found
    B[j]  $\leftarrow$  A[i]
  end
  A  $\leftarrow$  B // copy B into A
  // This gets a different arrangement of numbers in A, but they are
  // not necessarily sorted. Must check for that
  for i  $\leftarrow$  1 to n do
    if (A[i] > A[i + 1]) then failure
  end
  // This loop causes failure on any order except the correct one. It
  // forces the choice function to pick the right order.
end

```

Satisfiability (SAT)

Given k boolean variables $x_1, x_2, \dots, x_k$ and a boolean expression in conjunctive normal form (CNF):

$$E = (x_1 \vee x_2) \wedge (x_3 \vee x_4) \dots$$

The satisfiability problem is to determine if there is some assignment of truth values to the x_i 's that will make E true.

Deterministic algorithm: The simple approach is to try all combinations of the truth values for the x 's. This will take $O(2^k)$ for k x 's. There are no known algorithms of polynomial time.

Nondeterministic SAT

```

procedure NDSAT( E, k)
  for i  $\leftarrow$  1 to k do
     $x_i \leftarrow$  choice(true, false)
  end
  if E( $x_1, x_2, \dots, x_k$ ) then success
  else failure
  endif
end

```

Time taken by NDSAT is $O(k)$ for the loop, evaluation of E is proportional to the length of the formula.
 $\therefore$ polynomial time.

Definition of P: P is the set of all decision problems that can be solved by deterministic algorithms in polynomial time.

Definition of NP: NP is the set of all decision problems solvable by nondeterministic algorithms in polynomial time.

It is easy to see that $P \subseteq NP$. If a problem can be solved in polynomial time, then it can be verified in polynomial time.

It has not been proven whether $P = NP$ or $P \neq NP$.

However, Cook showed that if the SAT problem, which seems to be NP , is P , then $P = NP$.

If $SAT \in P$, then $P = NP$.

Every problem in NP reduces to SAT, i.e. there is no problem in NP that is harder than SAT. (See Garey & Johnson, "Computers & Intractability").

We say A reduces to B ($A \propto B$), if solving problem B tells us how to solve problem A . This indicates that A is not harder than B .

Definition: $A \propto B$ if an instance of A can be transformed into an instance of B in polynomial time, so that the answer to B is "yes" iff the answer to A is "yes".

Cook's Theorem (1971): Every problem in NP reduces to SAT.

Definition: A problem X is NP-complete if:

- $X \in NP$ and
- all NP problems reduce to X .

Definition: A problem X is NP-hard iff $SAT \propto X$.

Definition: A problem X is NP-complete iff X is NP-hard and $X \in NP$.

Note: It is possible for a problem to be NP-hard but not in NP .

Example: The Halting problem is NP-hard but not NP-complete.

Halting problem: is there a deterministic decision algorithm that can examine a program P and decide whether the program will ever stop or will run forever for a given input I ? The answer is no.

Proof: Suppose we had such an algorithm $H(P, I)$ that would return *true* if P eventually halts on input I , else *false*.

Now write the following program X :

```
if  $H(P, I)$  then while true do end;  
else stop
```

Now run this program using itself as the problem P .

If the halting function decides that program X will halt, X enters an infinite loop.

If the halting function decides that program X never stops, X promptly stops.

Therefore, there is no algorithm H that does this, so clearly $H \notin NP$, and so is not NP-complete.

Now, suppose we construct an algorithm that tries all 2^k possible truth values to decide whether a formula E can ever be true. If it finds a set of values, this algorithm halts. If it exhausts all possibilities without finding any set, it enters an infinite loop.

If we had a deterministic polynomial algorithm for the Halting problem, we could use it to solve the Satisfiability problem in polynomial time, so

Satisfiability $\propto$ Halting problem

$\therefore$ Halting problem is NP-hard but not NP-complete.